# CS253 Assignment 1 Report
# Memory-Efficient Versioned File Indexer Using C++

Chaniyara Yug
Roll Number: 240299

## 1 Introduction / Overview

This assignment implements a memory-efficient versioned file indexer in C++. The program reads very large text files using a fixed-size buffer and builds a word-level index without loading the entire file into memory.

The program supports:

- Word frequency query
- Top-K frequent words
- Difference between two versions

## 2 Design and Approach

The program follows object-oriented design.

Steps:

1. Parse command line arguments
2. Create QueryProcessor object
3. Read file using fixed buffer
4. Tokenize words
5. Build index using unordered_map
6. Execute query

Memory usage depends only on number of unique words.

## 3 Classes Used

### FileBufferReader

Reads file using fixed-size buffer.

- Buffer size between 256 KB and 1024 KB
- Reads file incrementally

### Tokenizer

Converts characters to words.

- Case insensitive
- Handles split words

### VersionIndex

Stores word frequencies using

```
unordered_map<string,int>
```

Also demonstrates function overloading.

### QueryProcessor

Abstract base class.

Derived classes:

- WordQueryProcessor
- TopKQueryProcessor
- DiffQueryProcessor

Demonstrates inheritance and polymorphism.

## 4 Fixed Size Buffer Processing

File is processed in chunks.

Actual code used:

```
while (!reader.eof()) {
    auto bytesRead = reader.readChunk(buffer);
    if (bytesRead > 0) {
        tokenizer.tokenizeChunk(buffer, bytesRead, index);
    }
}
tokenizer.finalize(index);
```

Buffer size never changes. Entire file is never loaded into memory.

## 5 Assumptions

- Words are alphanumeric
- Case insensitive matching
- Buffer size between 256 and 1024 KB
- Each file is one version

## 6 Observations

- Memory depends on unique words only
- Larger buffer gives faster execution
- Diff query is slower because two files are processed

## 7 Compilation and Run Commands

Compile:

```
g++ 240299_CHANIYARA.cpp -O2 -o analyzer
```

Run commands used:

Word query (error)

```
./analyzer --file test_logs.txt --version v1 --buffer 256 --query word
   --word error
```

Word query (DEVOPS)

```
./analyzer --file test_logs.txt --version v1 --buffer 256 --query word
   --word DEVOPS
```

Word query (devops)

```
./analyzer --file test_logs.txt --version v1 --buffer 256 --query word
   --word devops
```

Top-10 query

```
./analyzer --file test_logs.txt --version v1 --buffer 256 --query top --
   top 10
```

Diff query (error)

```
./analyzer --file1 test_logs.txt --version1 v1 --file2 verbose_logs.txt
   --version2 v2 --buffer 256 --query diff --word error
```

Diff query (request)

```
./analyzer --file1 test_logs.txt --version1 v1 --file2 verbose_logs.txt
   --version2 v2 --buffer 256 --query diff --word request
```

## 8 Conclusion

The program satisfies all requirements:

- Fixed-size buffer
- Object oriented design
- Inheritance and polymorphism
- Function overloading
- Exception handling

- Template usage
- Command line input

Memory usage is independent of file size.